Bayesian Logistic Regression

DS 4420 Final Project

Load Libraries

```r
library(ggplot2)
library(dplyr)
library(brms)
```

Load and Prepare Data

```r
df <- read.csv("../data/boston_crime_combined.csv")

# clean up SHOOTING column
df$SHOOTING <- ifelse(df$SHOOTING %in% c(1, "Y"), 1, 0)
df$SHOOTING[is.na(df$SHOOTING)] <- 0

# keep only relevant columns
df <- df[, c("SHOOTING", "MONTH", "HOUR", "DAY_OF_WEEK", "DISTRICT")]

# data is already cleaned in data_prep, just double check
df <- df[complete.cases(df), ]

# encode DAY_OF_WEEK as numeric
day_levels <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")
df$DAY_OF_WEEK <- as.numeric(factor(df$DAY_OF_WEEK, levels = day_levels))

# brms needs SHOOTING as factor for bernoulli family
df$SHOOTING <- as.factor(df$SHOOTING)

cat("Size after cleaning:", nrow(df), "\n")
```

```
## Size after cleaning: 899980
```

```r
cat("Any NAs:", anyNA(df), "\n")
```

```
## Any NAs: FALSE
```

```r
cat("Shooting distribution:\n")
```

```
## Shooting distribution:
```

```r
print(table(df$SHOOTING))
```

```
##
##      0      1
## 893278   6702
```

Undersample to Balance Classes

```r
set.seed(42)

shooting_yes <- df[df$SHOOTING == 1, ]
shooting_no  <- df[df$SHOOTING == 0, ]

# randomly sample same number of non-shooting as shooting
shooting_no_sampled <- shooting_no[sample(nrow(shooting_no), nrow(shooting_yes)), ]

df_balanced <- rbind(shooting_yes, shooting_no_sampled)
df_balanced <- df_balanced[sample(nrow(df_balanced)), ]

cat("Balanced dataset size:", nrow(df_balanced), "\n")
```

```
## Balanced dataset size: 13404
```

```r
cat("Shooting distribution after undersampling:\n")
```

```
## Shooting distribution after undersampling:
```

```r
print(table(df_balanced$SHOOTING))
```

```
##
##    0    1
## 6702 6702
```

Train / Test Split

```r
set.seed(42)

n <- nrow(df_balanced)
train_idx <- sample(1:n, size = floor(0.8 * n))

df_train <- df_balanced[train_idx, ]
df_test  <- df_balanced[-train_idx, ]

cat("Train size:", nrow(df_train), "\n")
```

```
## Train size: 10723
```

```r
cat("Test size:", nrow(df_test), "\n")
```

```
## Test size: 2681
```

Bayesian Logistic Regression via brms

```r
# check default priors for our model
def_prior <- default_prior(
  SHOOTING ~ MONTH + HOUR + DAY_OF_WEEK + DISTRICT,
  data   = df_train,
  family = bernoulli("logit")
)

print(def_prior)
```

```
##                 prior     class        coef group resp dpar nlpar lb ub tag
##                (flat)         b
##                (flat)         b DAY_OF_WEEK
##                (flat)         b DISTRICTA15
##                (flat)         b  DISTRICTA7
##                (flat)         b  DISTRICTB2
##                (flat)         b  DISTRICTB3
##                (flat)         b DISTRICTC11
##                (flat)         b  DISTRICTC6
##                (flat)         b DISTRICTD14
##                (flat)         b  DISTRICTD4
##                (flat)         b DISTRICTE13
##                (flat)         b DISTRICTE18
##                (flat)         b  DISTRICTE5
##                (flat)         b        HOUR
##                (flat)         b       MONTH
##  student_t(3, 0, 2.5) Intercept
##        source
##       default
##  (vectorized)
##  (vectorized)
##  (vectorized)
##  (vectorized)
##  (vectorized)
##  (vectorized)
##  (vectorized)
##  (vectorized)
##  (vectorized)
##  (vectorized)
##  (vectorized)
##  (vectorized)
##  (vectorized)
##  (vectorized)
##       default
```

```r
# fit the model
set.seed(42)
brm_fit <- brm(
  SHOOTING ~ MONTH + HOUR + DAY_OF_WEEK + DISTRICT,
  family  = bernoulli("logit"),
  data    = df_train,
  prior   = def_prior,
  chains  = 4,
  iter    = 2000,
  warmup  = 500,
  cores   = 1,
  seed    = 42
)
```

```
## Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c
## using C compiler: ‘Apple clang version 17.0.0 (clang-1700.6.3.2)’
## using SDK: ‘MacOSX26.2.sdk’
## clang -arch arm64 -std=gnu2x -I"/Library/Frameworks/R.framework/Resources/include" -DNDEBUG   -I"/Li
## In file included from <built-in>:1:
## In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/StanHeade
```

```
## In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/RcppEigen/
## In file included from /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/RcppEigen/
## /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/RcppEigen/include/Eigen/src/Core
##   679 | #include <cmath>
##       |          ^~~~~~
## 1 error generated.
## make: *** [foo.o] Error 1
```

```
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.000292 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 2.92 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 1: Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 1: Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 1: Iteration:  501 / 2000 [ 25%]  (Sampling)
## Chain 1: Iteration:  700 / 2000 [ 35%]  (Sampling)
## Chain 1: Iteration:  900 / 2000 [ 45%]  (Sampling)
## Chain 1: Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 1: Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 1: Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1: Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 1: Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 1: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1:
## Chain 1:  Elapsed Time: 4.239 seconds (Warm-up)
## Chain 1:                4.026 seconds (Sampling)
## Chain 1:                8.265 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 0.000229 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 2.29 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2: Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 2: Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 2: Iteration:  501 / 2000 [ 25%]  (Sampling)
## Chain 2: Iteration:  700 / 2000 [ 35%]  (Sampling)
## Chain 2: Iteration:  900 / 2000 [ 45%]  (Sampling)
## Chain 2: Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 2: Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 2: Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 2: Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 2: Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 2: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2:
```

```
## Chain 2:  Elapsed Time: 4.343 seconds (Warm-up)
## Chain 2:                3.459 seconds (Sampling)
## Chain 2:                7.802 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 0.000238 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 2.38 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3: Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 3: Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 3: Iteration:  501 / 2000 [ 25%]  (Sampling)
## Chain 3: Iteration:  700 / 2000 [ 35%]  (Sampling)
## Chain 3: Iteration:  900 / 2000 [ 45%]  (Sampling)
## Chain 3: Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 3: Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 3: Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3: Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 3: Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 3: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3:
## Chain 3:  Elapsed Time: 4.142 seconds (Warm-up)
## Chain 3:                3.914 seconds (Sampling)
## Chain 3:                8.056 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.00023 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 2.3 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4: Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 4: Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 4: Iteration:  501 / 2000 [ 25%]  (Sampling)
## Chain 4: Iteration:  700 / 2000 [ 35%]  (Sampling)
## Chain 4: Iteration:  900 / 2000 [ 45%]  (Sampling)
## Chain 4: Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 4: Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 4: Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 4: Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 4: Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 4: Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4:
## Chain 4:  Elapsed Time: 4.598 seconds (Warm-up)
## Chain 4:                3.865 seconds (Sampling)
## Chain 4:                8.463 seconds (Total)
## Chain 4:
```

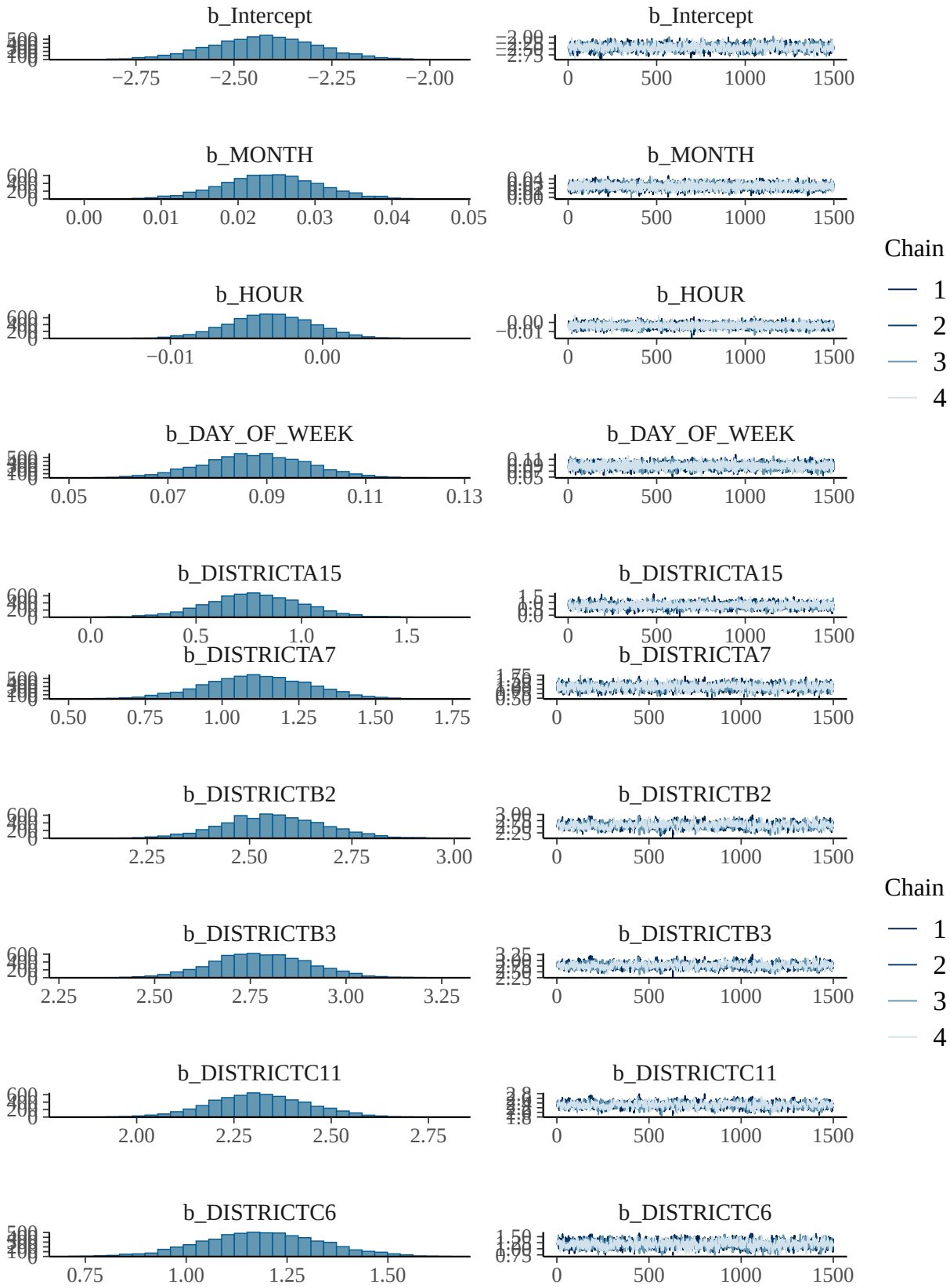
Model Summary

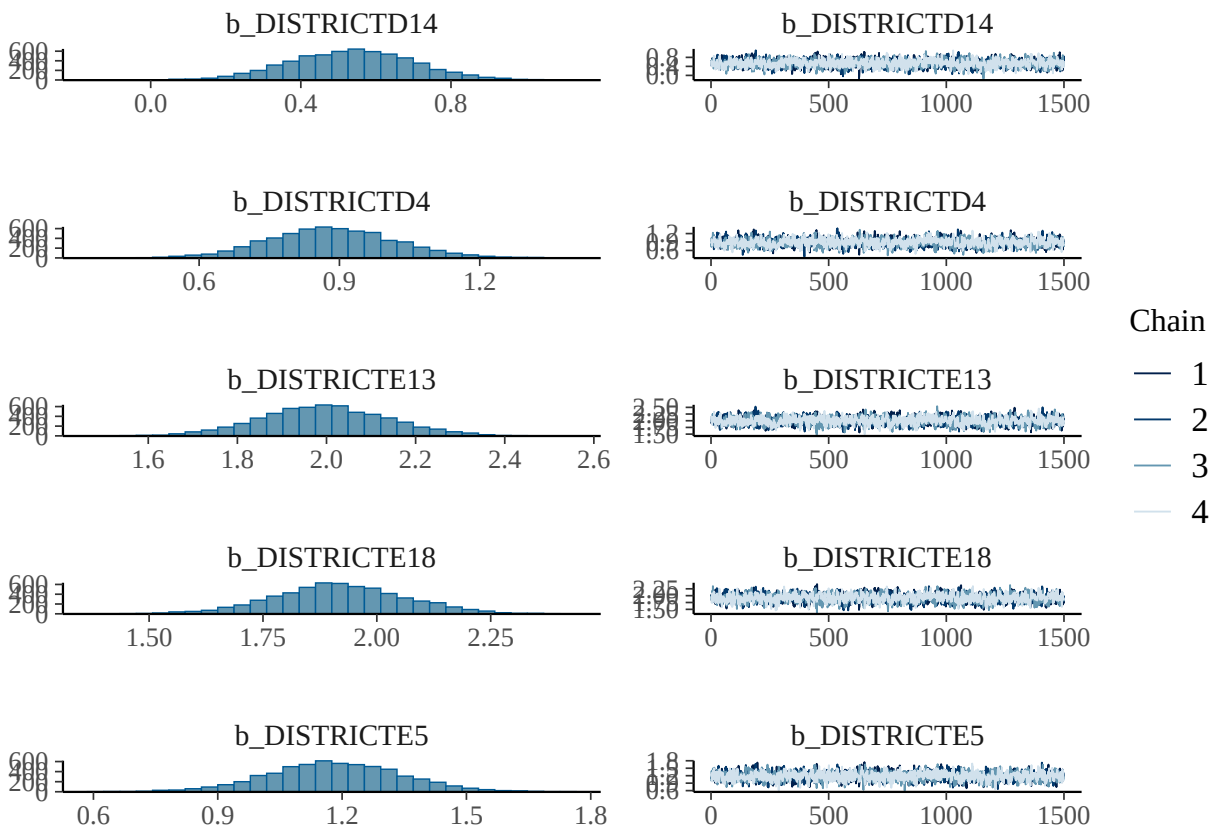
```r
summary(brm_fit)
```

```
##  Family: bernoulli
##   Links: mu = logit
## Formula: SHOOTING ~ MONTH + HOUR + DAY_OF_WEEK + DISTRICT
##    Data: df_train (Number of observations: 10723)
##   Draws: 4 chains, each with iter = 2000; warmup = 500; thin = 1;
##          total post-warmup draws = 6000
##
## Regression Coefficients:
##             Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## Intercept      -2.42      0.14    -2.69    -2.16 1.00     1380     2264
## MONTH           0.02      0.01     0.01     0.04 1.00     8839     4980
## HOUR           -0.00      0.00    -0.01     0.00 1.00    12919     4824
## DAY_OF_WEEK     0.09      0.01     0.07     0.11 1.00     6459     4578
## DISTRICTA15     0.78      0.22     0.34     1.20 1.00     2388     3195
## DISTRICTA7      1.11      0.17     0.77     1.45 1.00     1842     2569
## DISTRICTB2      2.56      0.12     2.32     2.80 1.00     1155     1543
## DISTRICTB3      2.78      0.13     2.54     3.02 1.00     1173     1687
## DISTRICTC11     2.32      0.13     2.07     2.56 1.00     1151     1773
## DISTRICTC6      1.19      0.15     0.90     1.48 1.00     1434     2399
## DISTRICTD14     0.54      0.16     0.22     0.86 1.00     1663     2508
## DISTRICTD4      0.88      0.14     0.62     1.16 1.00     1291     1935
## DISTRICTE13     2.00      0.14     1.72     2.28 1.00     1390     1962
## DISTRICTE18     1.92      0.14     1.63     2.20 1.00     1367     1982
## DISTRICTE5      1.18      0.16     0.86     1.50 1.00     1705     2535
##
## Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

Trace Plots

```r
# trace plots to check convergence
plot(brm_fit, ask = FALSE)
```


b_Intercept
b_MONTH
b_HOUR
b_DAY_OF_WEEK
b_DISTRICTA15
b_DISTRICTA7
b_DISTRICTB2
b_DISTRICTB3
b_DISTRICTC11
b_DISTRICTC6
Chain
1
2
3
4



Posterior Summaries

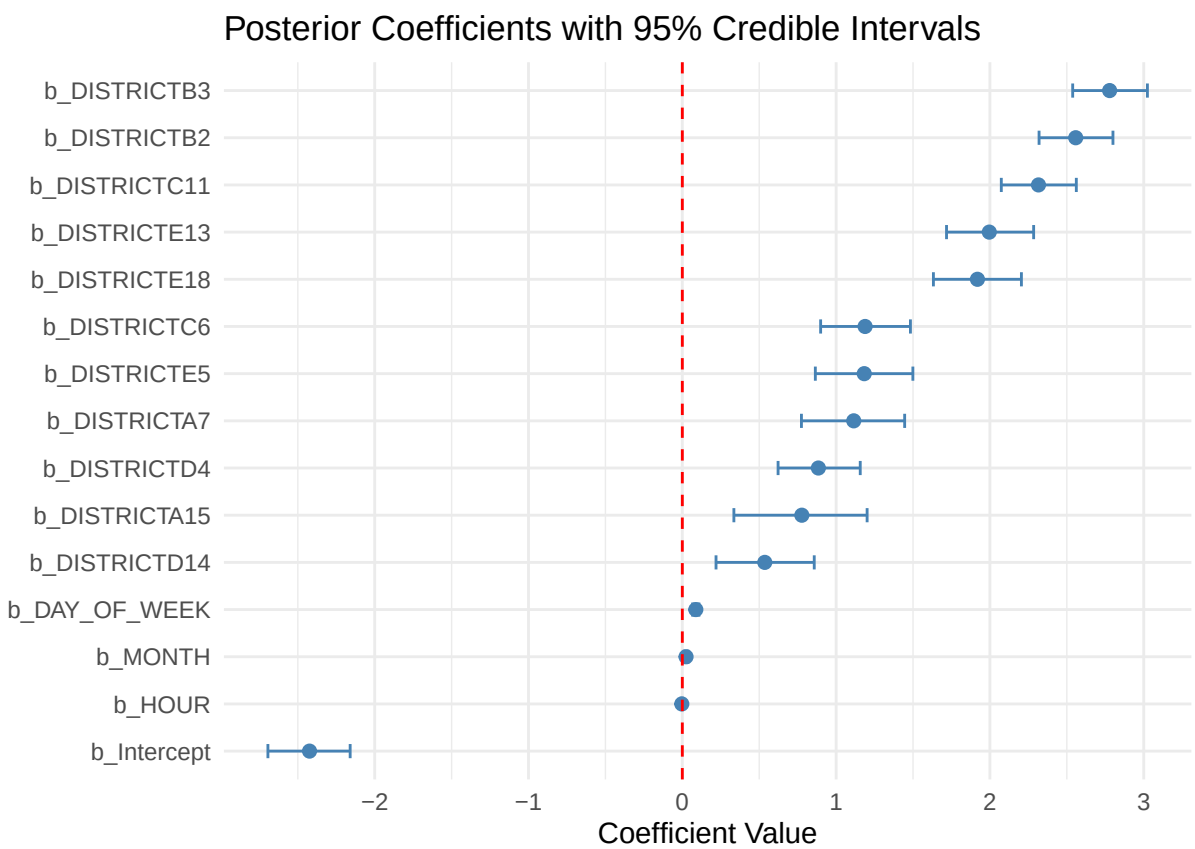
```
# extract posterior draws
post_draws <- as.data.frame(brm_fit)

# get coefficient names
coef_cols <- grep("^b_", names(post_draws), value = TRUE)

# posterior mean and 95% credible intervals
post_mean <- colMeans(post_draws[, coef_cols])
post_ci   <- apply(post_draws[, coef_cols], 2, quantile, probs = c(0.025, 0.975))

summary_df <- data.frame(
  coefficient = coef_cols,
  mean        = round(post_mean, 4),
  lower_95    = round(post_ci[1, ], 4),
  upper_95    = round(post_ci[2, ], 4)
)

# flag coefficients whose CI doesn't include 0
summary_df$significant <- ifelse(
  summary_df$lower_95 > 0 | summary_df$upper_95 < 0, "*", ""
)

print(summary_df)
```

```
##                  coefficient    mean lower_95 upper_95 significant
```

```
## b_Intercept     b_Intercept -2.4238  -2.6943  -2.1597          *
## b_MONTH             b_MONTH  0.0242   0.0117   0.0367          *
## b_HOUR               b_HOUR -0.0034  -0.0090   0.0020
## b_DAY_OF_WEEK b_DAY_OF_WEEK  0.0877   0.0672   0.1080          *
## b_DISTRICTA15 b_DISTRICTA15  0.7771   0.3353   1.2015          *
## b_DISTRICTA7   b_DISTRICTA7  1.1146   0.7745   1.4457          *
## b_DISTRICTB2   b_DISTRICTB2  2.5578   2.3195   2.8000          *
## b_DISTRICTB3   b_DISTRICTB3  2.7784   2.5381   3.0240          *
## b_DISTRICTC11 b_DISTRICTC11  2.3158   2.0740   2.5618          *
## b_DISTRICTC6   b_DISTRICTC6  1.1884   0.8993   1.4831          *
## b_DISTRICTD14 b_DISTRICTD14  0.5362   0.2191   0.8578          *
## b_DISTRICTD4   b_DISTRICTD4  0.8846   0.6227   1.1569          *
## b_DISTRICTE13 b_DISTRICTE13  1.9959   1.7179   2.2843          *
## b_DISTRICTE18 b_DISTRICTE18  1.9189   1.6326   2.2048          *
## b_DISTRICTE5   b_DISTRICTE5  1.1828   0.8648   1.4990          *
```

Posterior Coefficient Plot

```r
ggplot(summary_df, aes(x = mean, y = reorder(coefficient, mean))) +
  geom_point(color = "steelblue", size = 2) +
  geom_errorbarh(aes(xmin = lower_95, xmax = upper_95), height = 0.3, color = "steelblue") +
  geom_vline(xintercept = 0, color = "red", linetype = "dashed") +
  labs(title = "Posterior Coefficients with 95% Credible Intervals",
       x = "Coefficient Value", y = "") +
  theme_minimal()
```



Classification on Test Set

```r
# get posterior predictive probabilities on test set
prob_test <- posterior_epred(brm_fit, newdata = df_test)

# use posterior mean probability for classification
prob_mean <- colMeans(prob_test)
pred_test <- ifelse(prob_mean > 0.5, 1, 0)
y_test    <- as.numeric(as.character(df_test$SHOOTING))

conf_mat <- table(Predicted = pred_test, Actual = y_test)
print(conf_mat)
```

```
##          Actual
## Predicted    0    1
##         0  692  302
##         1  583 1104
```

```r
accuracy  <- mean(pred_test == y_test)
cat("Test accuracy:", round(accuracy, 4), "\n")
```

```
## Test accuracy: 0.6699
```

```r
tp <- conf_mat[2, 2]
fp <- conf_mat[2, 1]
fn <- conf_mat[1, 2]

precision <- tp / (tp + fp)
recall    <- tp / (tp + fn)
f1        <- 2 * precision * recall / (precision + recall)

cat("Precision:", round(precision, 4), "\n")
```

```
## Precision: 0.6544
```

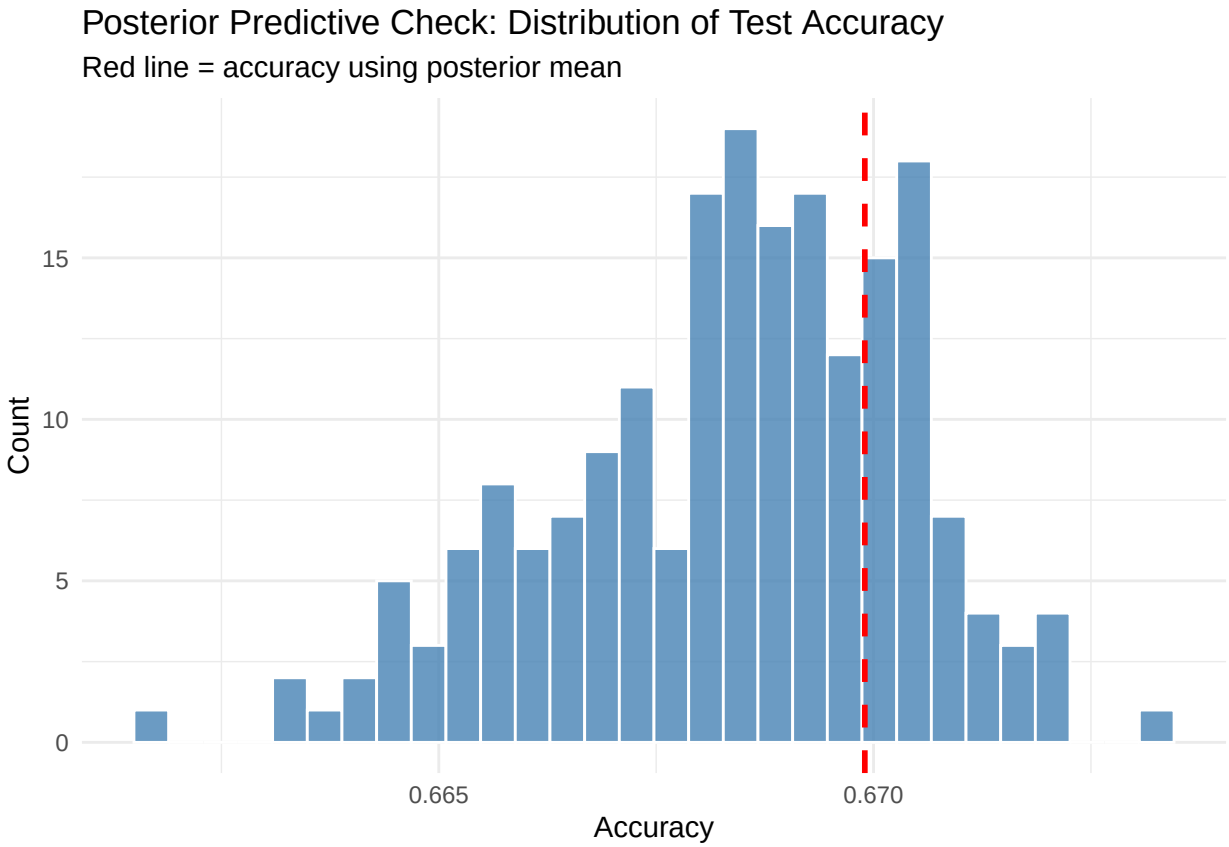
```r
cat("Recall:",    round(recall, 4), "\n")
```

```
## Recall: 0.7852
```

```r
cat("F1 Score:",  round(f1, 4), "\n")
```

```
## F1 Score: 0.7139
```

Posterior Predictive Check

```r
# distribution of test accuracy across posterior samples
n_check  <- 200
ppc_acc  <- numeric(n_check)

for (i in 1:n_check) {
  preds      <- ifelse(prob_test[i, ] > 0.5, 1, 0)
  ppc_acc[i] <- mean(preds == y_test)
}

ggplot(data.frame(accuracy = ppc_acc), aes(x = accuracy)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white", alpha = 0.8) +
  geom_vline(xintercept = accuracy, color = "red", linetype = "dashed", linewidth = 1) +
```

```r
labs(title = "Posterior Predictive Check: Distribution of Test Accuracy",
     subtitle = "Red line = accuracy using posterior mean",
     x = "Accuracy", y = "Count") +
theme_minimal()
```